

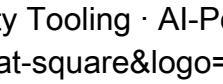
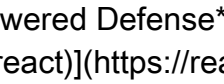
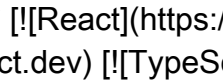
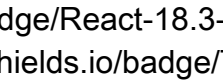
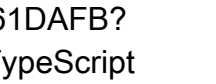
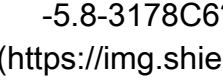
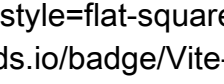
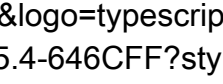
 SafeByte ### Enterprise-Grade Cybersecurity Platform **Penetration Testing · Threat Detection · Security Tooling · AI-Powered Defense**         

Table of Contents

- [About SafeByte](#)
- [Platform Overview](#)
- [Tech Stack](#)
- [Project Structure](#)
- [Web Tools — Deep Dive](#)
- [Malware Detector](#)
- [Vulnerability Assessment Tool](#)
- [FraudEye Scanner](#)
- [CryptoTrace AI](#)
- [System Auditing Tools \(Desktop\)](#)
- [Password Manager](#)
- [File System Crawler](#)
- [USB Detection Tool](#)
- [Services](#)
- [Site Pages](#)
- [AI Chatbot](#)
- [Environment Variables](#)
- [Getting Started](#)
- [API Integrations](#)
- [Contact](#)

About SafeByte

SafeByte is a cybersecurity consultancy and tooling company founded with a single mission: make enterprise-grade security accessible to every organisation, regardless of size. We combine hands-on offensive security expertise with a suite of proprietary tools that run directly in the browser — no server, no data leakage, no friction.

Our team holds certifications including OSCP, CISSP, CEH, CISM, and AWS Security Specialty , with over 4,000 security assessments delivered across financial services, healthcare, critical infrastructure, and technology sectors.

“SafeByte transformed our security posture. Their team identified critical vulnerabilities we'd missed for years and deployed solutions within days.” — Rachel Matsuda, CTO · Vaultline Technologies

🌐 Platform Overview

This repository is the SafeByte V2 web platform — a full-stack cybersecurity web application built with React + TypeScript. It serves as:

- 1. The public-facing company website — services, about, and contact pages
- 2. A browser-based security tooling suite — four tools that run entirely client-side
- 3. A download hub for desktop (macOS + Windows) security applications
- 4. An AI-powered chatbot for visitor Q&A

All browser-based tools perform analysis entirely within the user's browser . No file, address, or scan result is ever sent to SafeByte's servers.

🔧 Tech Stack

Frontend

Technology	Version	Purpose
React	18.3	UI framework
TypeScript	5.8	Type safety across the entire codebase
Vite	5.4	Build tool + dev server (port 8080)
Tailwind CSS	3.4	Utility-first styling
shadcn/ui	latest	Accessible, composable UI component library
Radix UI	various	Headless primitives powering shadcn
React Router v6	6.30	Client-side routing
TanStack Query	5.83	Server state management (ready for API expansions)
Lucide React	0.462	Icon library
Recharts	2.15	Chart components
Sonner	1.7	Toast notifications
EmailJS	4.4	Contact form email delivery (no backend needed)
next-themes	0.3	Theme management

Desktop Tools (Python)

Technology	Purpose
Python 3.10+	Runtime for all desktop tools
Rich	Terminal UI (tables, panels, progress bars)
SQLite3	Local encrypted database for vault + registry
Cryptography (Fernet)	AES-128 encryption for password vault
PBKDF2-HMAC-SHA256	Key derivation from master password
PyOTP	TOTP / Google Authenticator integration
Requests	HTTP API calls (VirusTotal, Etherscan, etc.)
python-dotenv	Environment variable management
ReportLab	PDF report generation

APIs Used

API	Tool	Purpose
VirusTotal v3	Malware Detector, FraudEye	File hash lookup, URL reputation
NVD (NIST)	VulnHawk	CVE database queries
FIRST.org EPSS	VulnHawk	Exploitation probability scores
Etherscan v2	CryptoTrace AI	Ethereum balance + transactions
Blockchain.info	CryptoTrace AI	Bitcoin balance + transactions
TronGrid	CryptoTrace AI	Tron balance lookup
EmailJS	Contact Form	Email delivery without a backend
Web Audio API	FraudEye	Browser-native audio analysis (deepfake detection)
Web Crypto API	Malware Detector	SHA-256 hashing in-browser

Project Structure

```

safebyte-v2/
├── public/
│   └── downloads/                                # Desktop tool binaries (.zip / .exe)
│       ├── SafeByte-Password-Manager-macOS.zip
│       ├── SafeByte-Password-Manager-Windows.exe
│       ├── SafeByte-File-Crawler-macOS.zip
│       ├── SafeByte-File-Crawler-Windows.exe
│       ├── SafeByte-USB-Monitor-macOS.zip
│       ├── SafeByte-USB-Monitor-Windows.exe
│       ├── password_manager_user_guide.pdf
│       └── file_listener_user_guide.pdf
├── src/
│   ├── App.tsx                                # Root component + React Router config
│   ├── main.tsx                              # Entry point
│   ├── index.css                             # Global styles + Tailwind directives
│   └── pages/
│       ├── Index.tsx                         # Home / landing page
│       ├── About.tsx                         # Company about page
│       ├── Services.tsx                      # Security services listing
│       ├── Tools.tsx                         # Tools catalogue (web + desktop)
│       ├── Contact.tsx                       # Contact form (EmailJS)
│       ├── MalwareScanner.tsx                # Web tool: Malware Detector
│       ├── VulnScanner.tsx                   # Web tool: Vulnerability Assessment
│       ├── FraudScanner.tsx                  # Web tool: FraudEye Scanner
│       ├── CryptoTrace.tsx                   # Web tool: CryptoTrace AI
│       └── NotFound.tsx                       # 404 page
│   └── components/
│       ├── Navbar.tsx                        # Fixed top navigation bar
│       ├── Footer.tsx                        # Site footer
│       ├── ChatBot.tsx                       # Floating AI chat assistant
│       └── ScrollReveal.tsx                   # IntersectionObserver animation
└── wrapper
    └── NavLink.tsx                           # Active-aware nav link

```

└─ ui/	# shadcn/ui component library
└─ (button, input, card, dialog, tabs, toast, ... 40+ components)	
└─ tools/	# Python desktop tool source code
└─ password_manager.py	
└─ malware_detector.py	
└─ list_files.py	
└─ usb_log.py	
└─ CryptoTraceAI/	
└─ startup.py	# Main CLI entry point
└─ .env	# API keys for desktop tool
└─ modules/	
└─ balance_checker.py	
└─ transaction_intelligence.py	
└─ wallet_detector.py	
└─ wallet_registry.py	
└─ registry_checker.py	
└─ risk_engine.py	
└─ case_manager.py	
└─ case_lookup.py	
└─ wallet_tracker.py	
└─ report_generator.py	
└─ pdf_report.py	
└─ dashboard.py	
└─ transaction_intelligence.py	
└─ hooks/	
└─ useChatbot.ts	# Chatbot response logic
└─ use-mobile.tsx	# Responsive breakpoint hook
└─ use-toast.ts	# Toast notification hook
└─ lib/	
└─ utils.ts	# Tailwind class merge utility (cn)
└─ .env.example	# Environment variable template
└─ .gitignore	
└─ package.json	
└─ vite.config.ts	
└─ tailwind.config.ts	
└─ tsconfig.json	
└─ components.json	# shadcn/ui configuration

Web Tools — Deep Dive

All four web tools live under `/tools/*` and run 100% in the browser.

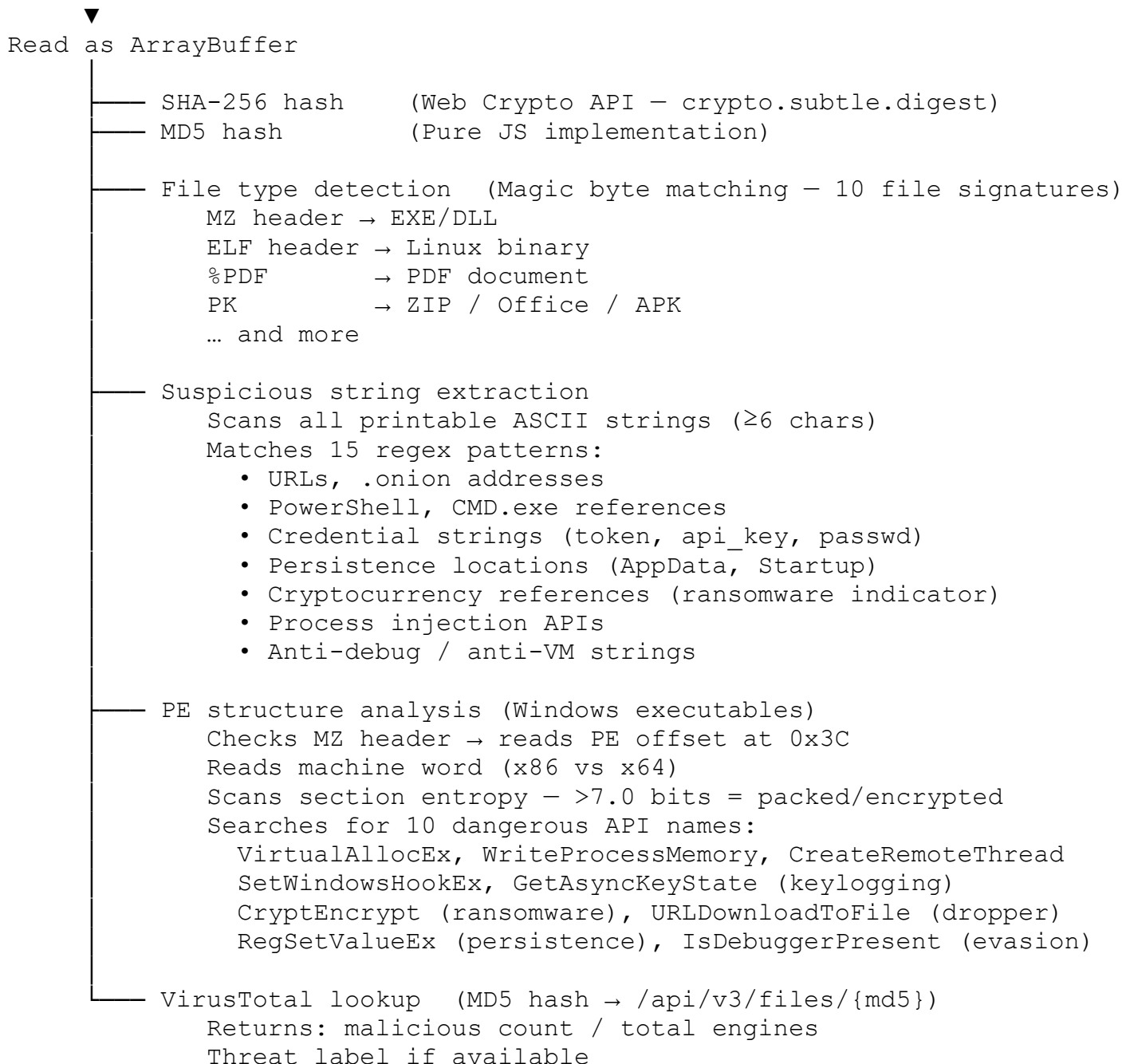
1. Malware Detector

Route: `/tools/malware-scanner`

Upload any file and get an instant static threat analysis without the file ever leaving your device.

How it works

File Upload / Drop
|



Verdict scoring

Factor	Points
≥5 VT engines detect malicious	+40
1–4 VT engines detect malicious	+20
VT suspicious detections	+10
Each high-risk string found	+8
Each dangerous API found	+10
High-entropy section detected	+10

Score thresholds: ≥40 = MALICIOUS · ≥20 = SUSPICIOUS · ≥5 = POTENTIALLY UNWANTED · <5 = CLEAN

2. Vulnerability Assessment Tool (VulnHawk)

Route: `/tools/vuln-scanner`

Enter your software stack, get a full CVE analysis with real exploitation probabilities and a prioritised fix roadmap.

How it works

User enters software + version (e.g. nginx 1.18.0)



NVD API query (NIST National Vulnerability Database)
GET <https://services.nvd.nist.gov/rest/json/cves/2.0?keywordSearch={software version}&resultsPerPage=15>



For each CVE:
Extract CVSS score (tries v3.1 → v3.0 → v2.0 fallback)
Extract description, references, published date



EPSS fetch (FIRST.org Exploit Prediction Scoring System)
GET <https://api.first.org/data/v1/epss?cve={ids}>
Returns: exploitation probability (0-1) per CVE



Enrich each CVE:
Priority Score = CVSS × (EPSS × 10 + 1)
Plain-English rewrite (jargon → readable explanation)
Risk label: CRITICAL / HIGH / MEDIUM / LOW / NONE



Attack Surface Score (0-100):
CRITICAL CVE × 20
HIGH CVE × 10
MEDIUM CVE × 4
High EPSS (>10%) bonus × 8 per CVE
Capped at 100



Results displayed as:
Overview tab — score ring, severity breakdown, top priority fixes
Per-service — expandable CVE cards with plain-English + technical detail

Quick presets available

Web Server · SSH Server · LAMP Stack · Node.js App · Database

3. FraudEye Scanner

Route: /tools/fraud-scanner

Three fraud detection modes in one tool — QR codes, voice recordings, and URLs.

Mode A — QR Code Scanner

Detects UPI payment request scams.

Paste UPI string (upi://pay?...)



Parse URLSearchParams

Extract: pa (payee address), pn (payee name),
am (amount), mode (transaction type)



KEY CHECK: mode=02 → PAYMENT REQUEST
(Money is DEDUCTED from scanner, not received)



Risk levels:

mode=02 present → DANGER (payment request scam)

Suspicious fields → WARNING

Clean UPI → SAFE

Mode B — Voice Deepfake Detector

Uses the browser-native Web Audio API to analyse voice recordings for AI-synthesis markers — no model download required.

Upload audio file (MP3, WAV, M4A, OGG)



AudioContext.decodeAudioData()



Frame-by-frame analysis (2048-sample frames, 512-sample hop)
Extracts 5 acoustic features per frame:

Feature 1 — Zero Crossing Rate (ZCR)

AI voices: unnaturally smooth (low ZCR) OR buzzy (high ZCR)

Flag: ZCR < 2% or > 35% per frame → +20 suspicion

Feature 2 — Pitch Variance

Manual DFT on 256-point downsampled frame

Dominant frequency in 60-400 Hz vocal range

AI voices: unnaturally consistent pitch

Flag: variance < 120 Hz² with 5+ voice frames → +30 suspicion

Feature 3 — Energy Variance

Per-frame RMS energy

AI voices: suspiciously steady amplitude

Flag: variance < 0.0005 with active audio → +25 suspicion

Feature 4 — Silence Pattern

Silent frames (energy < 0.0001)

AI voices: odd silence distributions

Flag: silence ratio > 40% or < 5% → +15 suspicion

Feature 5 — Spectral Flatness

Geometric mean / arithmetic mean of frame energies

AI voices: too flat (synthetic) or too tonal

Flag: flatness > 80% or < 2% → +10 suspicion

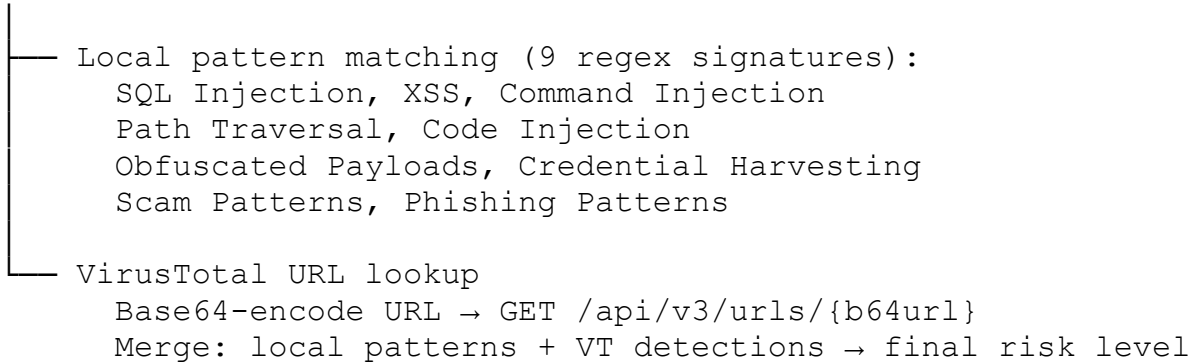


Suspicion score (0-100):

≥55 → DANGER (AI-generated voice)
≥30 → WARNING (suspicious)
<30 → SAFE (likely authentic)

Mode C — URL Safety Checker

Input URL



4. CryptoTrace AI

Route: /tools/crypto-trace

Trace any blockchain wallet — live balance, transaction intelligence, risk scoring, and OFAC registry cross-check.

Ported from the [CryptoTrace AI Python desktop tool](#) into a fully browser-based implementation.

Test Addresses

Use these real public addresses to test the tool:

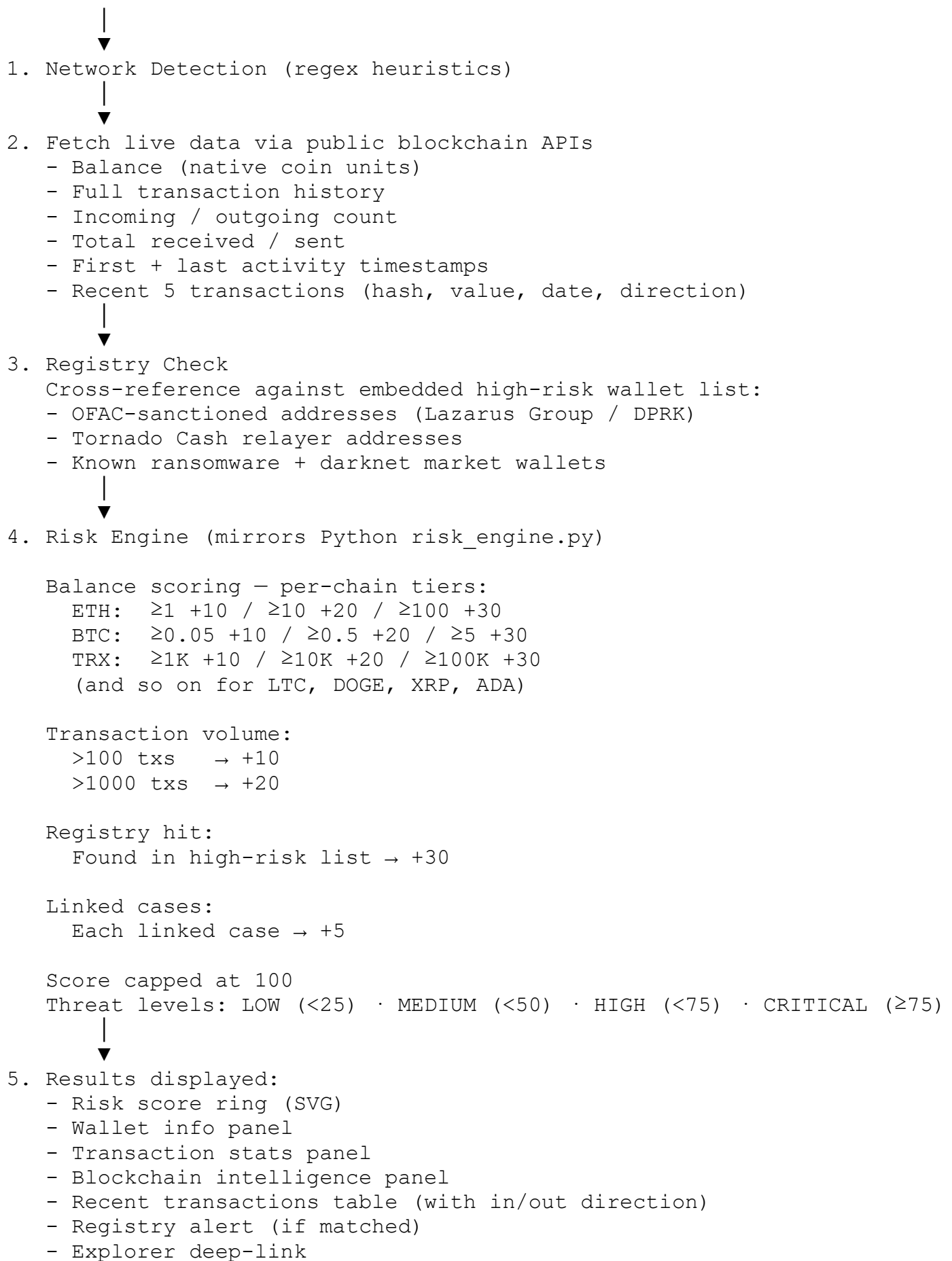
Network	Address	Notes
Ethereum	0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045	Vitalik Buterin's public wallet
Bitcoin	1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa	Genesis block address (Satoshi)
Tron	TKzxdSv2FZKQrEqkKVgp5DcwEXBEKMg2Ax	Binance public Tron hot wallet
OFAC Sanctioned	0x7F367cC41522cE07553e823bf3be79A889DEbe1B	Lazarus Group — triggers CRITICAL alert

Supported Networks

Network	Address Format	Balance API
Ethereum	0x + 40 hex chars	Etherscan v2 API
Bitcoin	1..., 3..., or bc1...	Blockchain.info
Tron	T + 33 alphanumeric	TronGrid API
Litecoin	ltc1..., L..., M...	Explorer link
Dogecoin	D + 33 alphanumeric	Explorer link
XRP	r + 24–33 alphanumeric	Explorer link
Cardano	addr1...	Explorer link

Analysis pipeline

Input: wallet address



🖥️ System Auditing Tools (Desktop)

Three standalone desktop applications available for both macOS and Windows.

1. Password Manager

Download: macOS [.zip](#) · Windows [.exe](#) · [User Guide PDF](#)

A GUI-based credential vault with two-factor protection.

Component	Implementation
Master password hashing	PBKDF2-HMAC-SHA256 (100,000 iterations)
Vault encryption	Fernet (AES-128-CBC + HMAC-SHA256)
2FA	TOTP via PyOTP (Google Authenticator compatible)
Storage	Encrypted SQLite database
Key handling	Encryption key is never stored in plaintext

Features: store, retrieve, update, delete credentials · generate strong passwords · vault lock after inactivity · export encrypted backup

2. File System Crawler

Download: macOS [.zip](#) · Windows [.exe](#) · [User Guide PDF](#)

Scan any directory for malware and suspicious files.

Crawl target directory recursively



For each file:

- Compute MD5 + SHA-256 hash
- Query VirusTotal (70+ antivirus engines)
- Local heuristics:
 - Dangerous filenames (passwords.txt, etc.)
 - Suspicious extensions (.vbs, .bat, .ps1, etc.)
 - Known persistence locations
- Verdict: SAFE / SUSPICIOUS / MALICIOUS



Live results table (Rich terminal UI)
Export: CSV or TXT report

3. USB Detection Tool

Download: macOS [.zip](#) · Windows [.exe](#)

Real-time background monitor for USB and external device events.

Runs silently in system tray



Monitors OS device events in real time



On device connect / disconnect:

- Plays beep alert
- Sends native OS notification
- Logs event to live table:
 - Timestamp · Device Name

Vendor ID · Product ID · Serial Number



Export events to CSV at any time

CryptoTrace AI (Desktop)

Run: `python startup.py` from `src/components/tools/CryptoTraceAI/`

The full desktop version of CryptoTrace with additional capabilities not available in the browser tool.

Module	Function
<code>wallet_detector.py</code>	Network detection + full analysis orchestration
<code>balance_checker.py</code>	Balance fetching for all 7 chains
<code>transaction_intelligence.py</code>	Full TX history via Etherscan, Blockchain.info, TronGrid, etc.
<code>risk_engine.py</code>	0–100 risk scoring with per-network balance tiers
<code>registry_checker.py</code>	SQLite registry lookup
<code>wallet_registry.py</code>	Add / remove / view high-risk wallet registry
<code>case_manager.py</code>	Create / view / close / delete investigation cases
<code>case_lookup.py</code>	Link wallets to existing cases
<code>wallet_tracker.py</code>	Watchlist management for monitored wallets
<code>report_generator.py</code>	Export investigation reports as TXT
<code>pdf_report.py</code>	Export investigation reports as PDF (ReportLab)
<code>dashboard.py</code>	System stats, API health, investigation statistics

Services

SafeByte offers the following professional security services (detailed at `/services`):

Service	Description
Penetration Testing	OSCP-certified simulated attacks — web, API, mobile, internal network, social engineering
Malware Analysis	Reverse engineering lab — static/dynamic analysis, behavioral sandboxing, IoC extraction
Risk Assessment	NIST CSF / ISO 27001 / CIS Controls mapping, threat modeling, executive risk reports
Network Security	Firewall/IDS/IPS configuration, network segmentation, traffic analysis, 24/7 SOC
Cloud Security	AWS/Azure/GCP config audits, IAM review, container security, compliance automation
Incident Response	24/7 rapid response, digital forensics, evidence preservation, post-incident review

Site Pages

Route	Component	Description
-------	-----------	-------------

/	Index.tsx	Landing page — hero, services overview, tools highlight, testimonial, CTA
/about	About.tsx	Company story, mission/vision, core values (Integrity, Precision, Vigilance, Partnership), certifications
/services	Services.tsx	Detailed service cards with feature lists
/tools	Tools.tsx	Tools catalogue split into System Auditing Tools (desktop) and Web Tools
/contact	Contact.tsx	Contact form with EmailJS delivery + auto-reply
/tools /malware-scanner	MalwareScanner.tsx	Web tool
/tools /vuln-scanner	VulnScanner.tsx	Web tool
/tools /fraud-scanner	FraudScanner.tsx	Web tool
/tools /crypto-trace	CryptoTrace.tsx	Web tool

AI Chatbot

A floating chat assistant is embedded on every page (`ChatBot.tsx`).

- Always visible via a floating button (bottom-right)
- Rule-based NLP via `useChatbot.ts` — keyword matching across services, tools, team, pricing, and contact topics
- Typing simulation — natural delay based on message length (600ms + 12ms/char, capped at 1500ms)
- Suggested questions shown on first open
- Unread badge when the bot replies while the window is closed
- Bold text rendering — `**text**` in bot responses renders as `<strong>`

Environment Variables

Copy `.env.example` to `.env` and fill in your keys. All variables are prefixed `VITE_` so Vite exposes them to the browser bundle.

```
cp .env.example .env
```

Variable	Tool	Where to Get
<code>VITE_VT_API_KEY</code>	Malware Detector, FraudEye	virustotal.com/gui/my-apikey
<code>VITE_NVD_API_KEY</code>	VulnHawk	nvd.nist.gov/developers
<code>VITE_EMAILJS_SERVICE_ID</code>	Contact Form	emailjs.com
<code>VITE_EMAILJS_TEMPLATE_ID</code>	Contact Form	EmailJS dashboard
<code>VITE_EMAILJS_AUTOREPLY_TEMPLATE_ID</code>	Contact Form	EmailJS dashboard
<code>VITE_EMAILJS_PUBLIC_KEY</code>	Contact Form	EmailJS dashboard

VITE_ETHERSCAN_API_KEY

CryptoTrace AI

etherscan.io/apis

VITE_TRONGRID_API_KEY

CryptoTrace AI

trongrid.io

Note: All tools degrade gracefully without API keys — VirusTotal lookups will be skipped, NVD queries fall back to public rate limits, and CryptoTrace uses Blockchain.info (no key needed) for Bitcoin.

Getting Started

Prerequisites

- [Bun](#) (recommended) or Node.js 18+

Install & Run

```
# Clone the repository
git clone https://github.com/aayush-ethclhkr/SafeByteV2.git
cd SafeByteV2
```

```
# Install dependencies
bun install          # or: npm install
```

```
# Set up environment variables
cp .env.example .env
# Edit .env and fill in your API keys
```

```
# Start the development server
bun run dev          # or: npm run dev
```

Open <http://localhost:8080> in your browser.

Build for Production

```
bun run build        # or: npm run build
bun run preview      # Preview the production build locally
```

CryptoTrace AI Desktop Tool

```
cd src/components/tools/CryptoTraceAI
```

```
# Create virtual environment
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
```

```
# Install dependencies
pip install -r requirements.txt
```

```
# Configure API keys
cp .env.example .env
# Edit .env with ETHERSCAN_API_KEY, TRONGRID_API_KEY, BLOCKFROST_API_KEY
```

```
# Run
python startup.py
```

API Integrations

API	Docs	Auth	Free Tier
VirusTotal v3	docs.virustotal.com	API key header	4 lookups/min
NVD (NIST)	nvd.nist.gov/developers	API key header	50 req/30s (with key)
FIRST.org EPSS	first.org/epss	None	Unlimited
Etherscan v2	docs.etherscan.io	API key query param	5 calls/sec
Blockchain.info	blockchain.info/api	None	Public
TronGrid	developers.tron.network	API key header	1000 req/day
EmailJS	emailjs.com/docs	Public key	200 emails/month

Contact

Email team.safebyte@gmail.com
Phone +91 89238 17932
Location Meerut, Uttar Pradesh, India
GitHub github.com/aayush-ethclhkr

Built with  by the SafeByte Team *Securing the Digital Future*